

VIT[®]
B H O P A L
www.vitbhopal.ac.in

Module 4

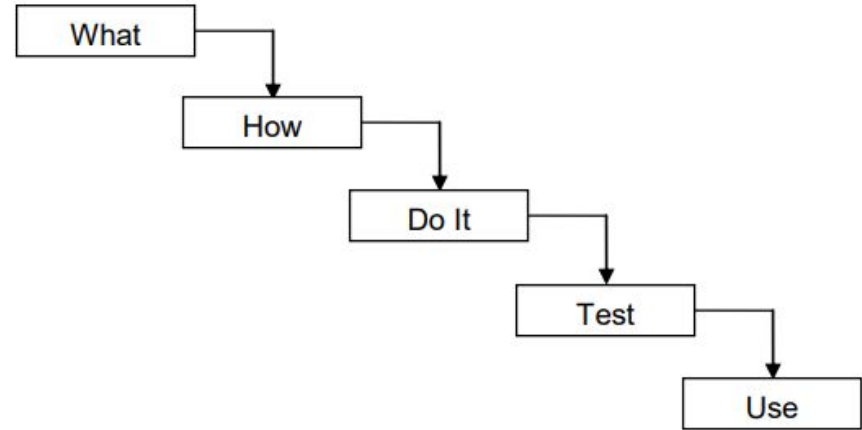
OBJECT ORIENTED LIFE CYCLE



- Overview of Object-Oriented Systems Development
- Object Oriented System Development Methodology
- Unified Approach
- Object Basics
- Object Oriented Systems Development Life Cycle
- Object Oriented Methodologies
 - Rumbaugh Methodology
 - Booch Methodology
 - Jacobson Methodology
 - Unified Approach.

Software development methodology

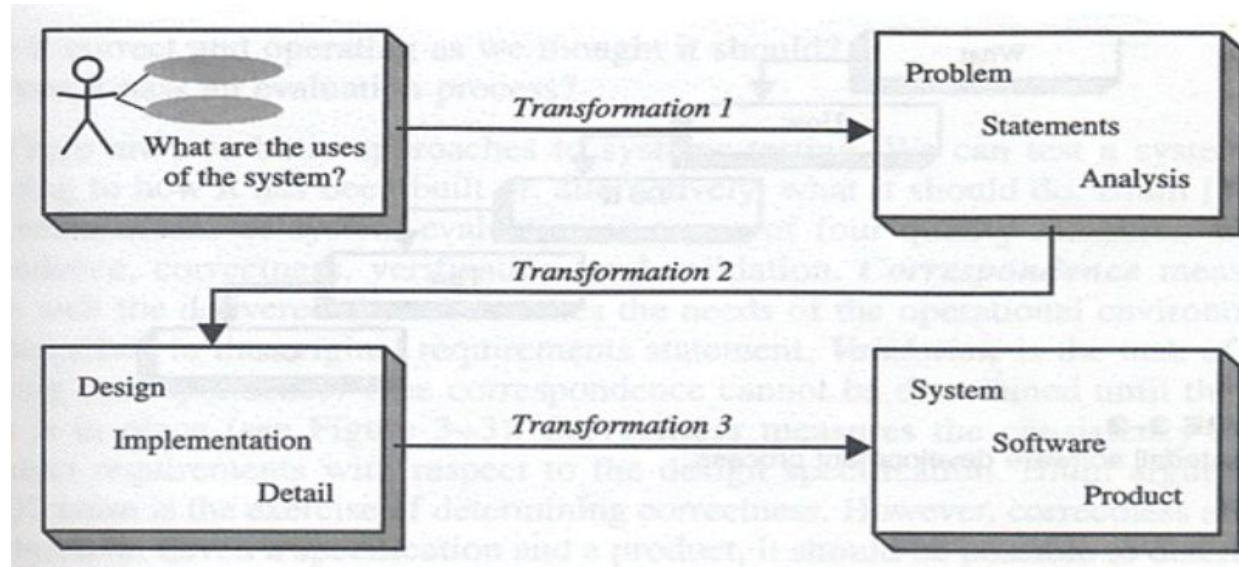
- A **software development methodology** is a series of processes that lead to the development of an application. A software development activity consists of:
 - System analysis
 - Modelling
 - Design
 - Implementation
 - Testing and
 - Maintenance



Waterfall Model- From 'what' to 'use'

Software Process- Transforming needs to software product

- **Software process:** It describes how the work is to be carried out to achieve the original goal based on the system requirement. Each process consists of number of steps and rules that should be performed during development.



Object Oriented Systems Development

- TWO ORTHOGONAL VIEWS OF SOFTWARE:
 - Traditional technique
 - Object oriented methodologies

Programs=Algorithm + Data Structures

TRADITIONAL APPROACH	OBJECT ORIENTED SYSTEM DEVELOPMENT
Collection of procedures(functions)	Combination of data and functionality
Focuses on function and procedures, different styles and methodologies for each step of process	Focuses on object, classes, modules that can be easily replaced, modified and reused.
Moving from one phase to another phase is complex.	Moving from one phase to another phase is easier.
Increases duration of project	decreases duration of project
Increases complexity	Reduces complexity and redundancy

Object Oriented Systems



Elements of Object-Oriented System

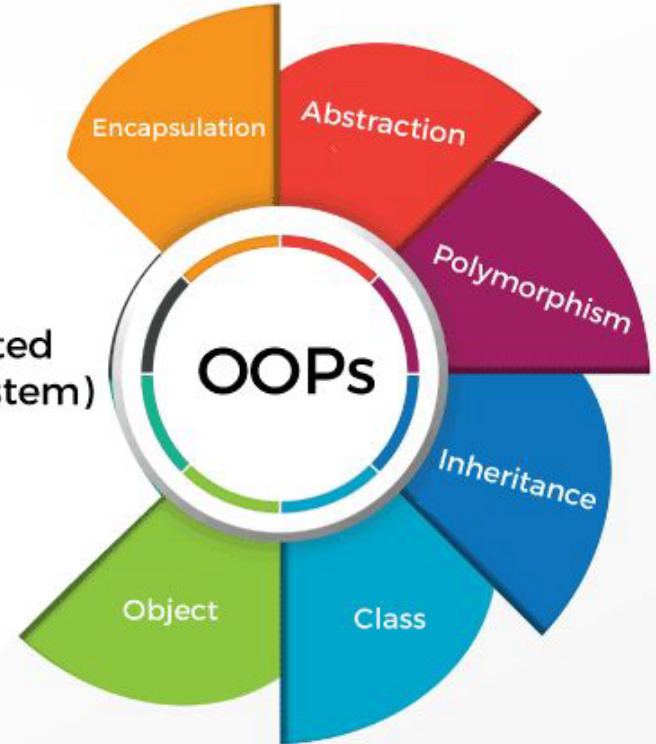
- **Objects** – An object is something that exists within problem domain and can be identified by data (attribute) or behavior. Example- student, patient, bank account
- **Attributes** – They describe information about the object.
- **Behavior** – It specifies what the object can do. It defines the operation performed on objects.
- **Class** – A class encapsulates the data and its behavior. Objects with similar meaning and purpose grouped together as class.
- **Methods** – Methods determine the behavior of a class. They are nothing more than an action that an object can perform.
- **Message** – A message is a function or procedure call from one object to another.

Object Oriented Concepts



- **Object-oriented approaches attempt to build models that mimic the “real world”**
- Object orientation emerged during the 1970s, particularly in programming languages like Smalltalk, C++ and Java

(Object - Oriented
Programming System)



Object Oriented Concepts

- **Classes and Objects:** In OOP, a class serves as a blueprint for creating objects. A class defines the attributes (data) and methods (functions) that its objects will possess. Objects are instances of classes, and they encapsulate both data and the operations that can be performed on that data.
- **Encapsulation:** Encapsulation is the principle of bundling data (attributes) and the methods that operate on that data within a single unit (an object). It helps hide the internal state of an object from the outside world, providing a level of abstraction and security.
- **Inheritance:** Inheritance allows one class to inherit the attributes and methods of another class. This promotes code reusability and allows for the creation of specialized classes that inherit common functionality from a base class.
- **Polymorphism:** Polymorphism enables objects of different classes to be treated as objects of a common superclass. This simplifies code by allowing the same interface to be used for different types of objects.

Benefits of Object Oriented Systems



- **Modularity:** OOP promotes the creation of modular code, making it easier to design, develop, and maintain software. Each object can be developed and tested independently, reducing complexity.
- **Reusability:** Inheritance and polymorphism facilitate code reuse, reducing redundancy and promoting efficient development practices.
- **Abstraction:** OOP allows developers to create abstract representations of real-world entities. This abstraction simplifies complex systems by focusing on essential characteristics and ignoring unnecessary details.
- **Encapsulation:** Encapsulation enhances security and robustness by controlling access to an object's data and ensuring that only designated methods can modify its state.
- **Ease of Maintenance:** Code organized using OOP principles is generally easier to maintain and debug. Changes to one part of the codebase are less likely to impact other parts if encapsulation is properly implemented.

Reasons for working of object oriented systems:



1. Higher level of abstraction:

- Object encapsulates both data and functions. So they work at higher level of abstraction. So designing, coding, testing and maintaining the system are much simpler.

2. Seamless transition among different phases of software development:

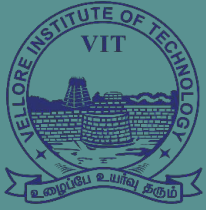
- We use the same language to talk about analysis, design, programming and database design, which makes clearer and robust system development.

3. Encouragement of good programming techniques:

- Changing one class has no impact on other classes, but there is communication between classes through interface. Promote clear design hence Implementation is easy

4. Promotion of reusability:

- Objects are reusable because they are modelled directly out of real world problem domain. The object orientation adds inheritance, which is a powerful technique that allows classes to built from each other.



VIT[®]
B H O P A L
www.vitbhopal.ac.in

Object Oriented Systems Development

Object Oriented Systems Development



Object oriented systems development: A use-case driven approach

The object-oriented software development life cycle (SDLC) consists of three macro processes: **object-oriented analysis, object-oriented design, and object-oriented implementation.**

Object oriented systems development includes these activities:

- ☐ Object-oriented analysis – use-case driven
- ☐ Object-oriented design
- ☐ Prototyping
- ☐ Component-based development
- ☐ Incremental testing

Object Oriented Systems Development



1. **Object-oriented analysis – use-case driven:**

- In this stage, the problem is formulated, user requirements are identified, and then a model is built based upon real-world objects.
- The analysis produces models on how the desired system should function and how it must be developed.
- The object-oriented analysis phase of software development is concerned with determining the system requirements and identifying classes and their relationships to other classes in the problem domain

2. **Object-Oriented design:**

- The goal of object-oriented design is to design the classes identified during the analysis phase and the user interface.
- During this phase, we identify and define additional objects and classes that support implementation of the requirements.
- Object-oriented design includes two main stages, namely, system design and object design.

Object Oriented Systems Development



3. Prototyping:

- A prototype is a version of a software product developed in the early stages of the product's life cycle for specific experimental purposes.
- A prototype enables you to fully understand how easy or difficult it will be implement some of the features of the system.

4. Implementation: Component-based development

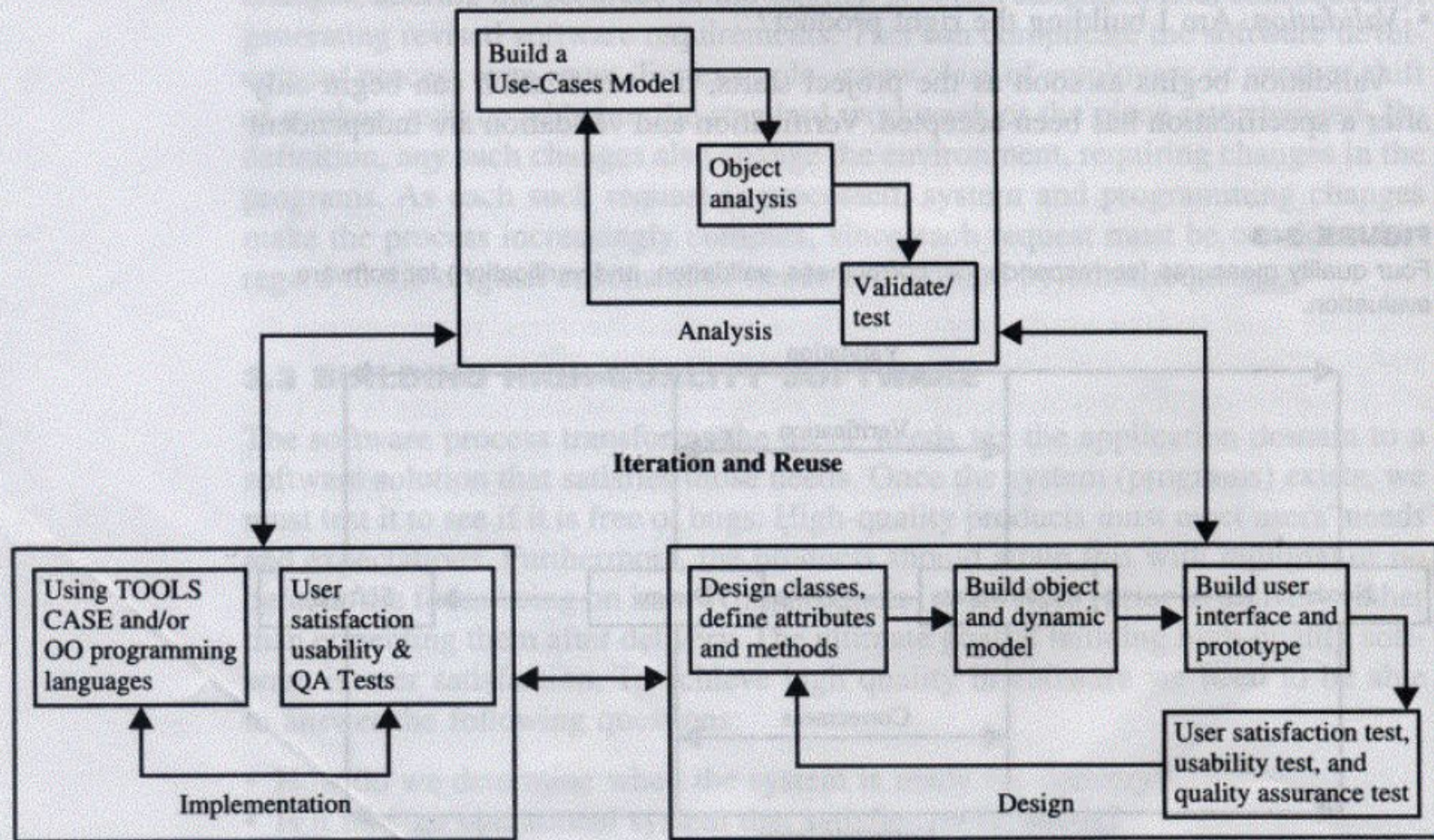
- CBD is an industrialized approach to the software development process.

5. Incremental testing

- It is used to test an application for bugs and its performance.

FIGURE 3-4

The object-oriented systems development approach. Object-oriented analysis corresponds to transformation 1; design to transformation 2, and implementation to transformation 3 of Figure 3-1.



Jacobson et al. life cycle model –
traceable design across development

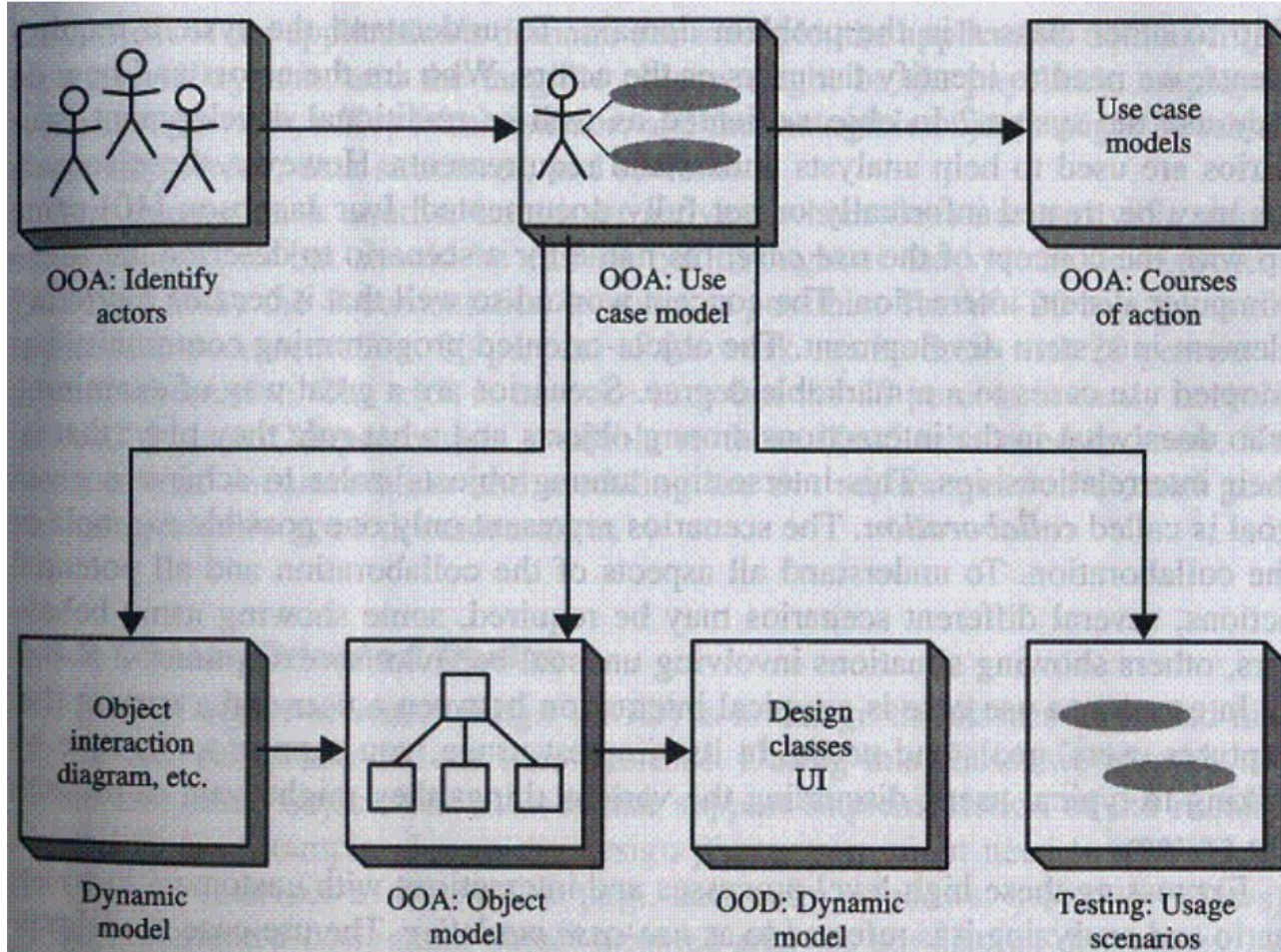
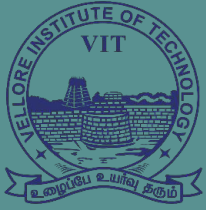


FIGURE 3-5

By following the life cycle model of Jacobson et al., we produce designs that are traceable across requirements, analysis, implementation, and testing.



VIT[®]
B H O P A L
www.vitbhopal.ac.in

Object Oriented Systems Development Methodolo

Object Oriented Systems Development Methodology



- Object oriented systems development methodology develops software by building objects that can be easily replaced , modified and reused.
- It is a system of cooperative and collaborating objects.
- It is a new system development approach, encouraging and facilitating re-use of software components.
- It employs international standard Unified Modelling Language (UML) from the Object Management Group (OMG).
- Object Oriented Methodology asks the analyst to determine
 - What the objects of the system are?,
 - What responsibilities and relationships an object has to do with the other objects? and
 - How they behave over time?

Object Oriented Systems Development Methodology



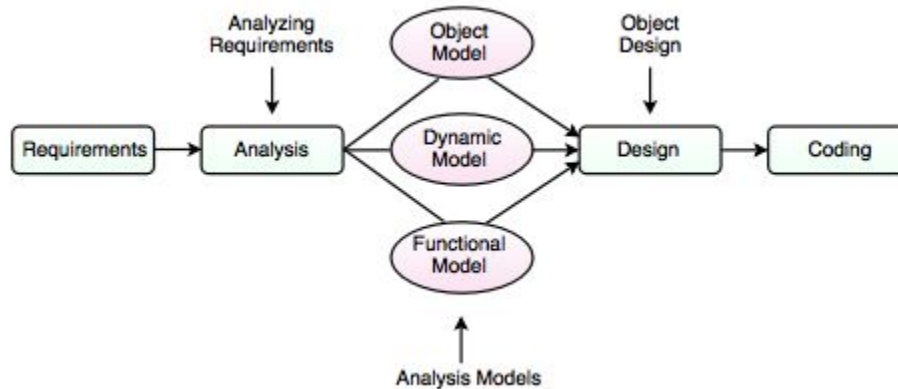
There are three types of Object Oriented Methodologies

- 1. Object Modelling Techniques (OMT)**
- 2. Object Process Methodology (OPM)**
- 3. Rational Unified Process (RUP)**

Object Oriented Systems Development Methodology

1. Object Modelling Techniques (OMT)

- It was one of the first object oriented methodologies and was introduced by Rumbaugh in 1991.
- OMT uses three different models that are combined in a way that is analogous to the older structured methodologies.



Object Oriented Systems Development Methodology



1. Object Modelling Techniques (OMT)

- **Analysis:** The main goal of the analysis is to build models of the world. The requirements of the users, developers and managers provide the information needed to develop the initial problem statement.
- **OMT Models**
 - **Object Model:** It depicts the object classes and their relationships as a class diagram, which represents the static structure of the system.
 - **Dynamic Model:** It captures the behavior of the system over time and the flow control and events in the Event-Trace Diagrams and State Transition Diagrams.
 - **Functional Model:** It describes the data transformations of the system using Data Flow diagrams.
- **Design:** In this phase, the details of the system analysis and system design are implemented. The objects identified in the system design phase are designed.

Object Oriented Systems Development Methodology



2. Object Process Methodology (OPM)

- It is also called as second generation methodology. It was first introduced in 1995.
- It has only one diagram that is the Object Process Diagram (OPD) which is used for modelling the structure, function and behavior of the system.
- It has a strong emphasis on modelling but has a weaker emphasis on process. It consists of three main processes:
 - I. **Initiating:** It determines high level requirements, the scope of the system and the resources that will be required.
 - II. **Developing:** It involves the detailed analysis, design and implementation of the system.
 - III. **Deploying:** It introduces the system to the user and subsequent maintenance of the system.

Object Oriented Systems Development Methodology

3. Rational Unified Process (RUP)

- It was developed in Rational Corporation in 1998.
- It consists of four phases which can be broken down into iterations.

I. Inception

II. Elaboration

III. Construction

IV. Transition

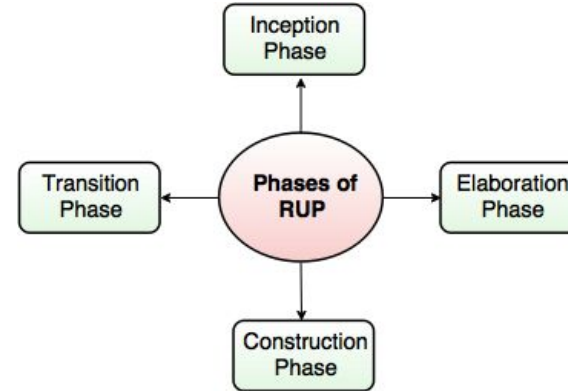


Fig. Phases of Rational Unified Process

Object Oriented Systems Development Methodology



3. Rational Unified Process (RUP)

- **1. Inception phase:** The developers define the scope of the project and its business case. It is important for the process to first formulate the scope of the project and also determine what resources will be needed.
- **2. Elaboration phase:** This phase is important to the RUP because it is here that developers analyze the risks associated with changing the scope of the project or adding new technologies along the way.
- **3. Construction phase:** The developers create the application design and source code. In this phase, the software is tested to determine if the project has met its goal. This phase produces the first external release of the software.
- **4. Transition phase:** The developers deliver the system to users. Final adjustment can be based on user feedback, usability or installation issues.

Benefits of Object Oriented Methodology

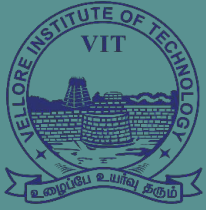


- **Simplicity:** The software object's model complexity is reduced and the program structure is very clear.
- **Reusability:** It makes easy to reuse the code in a new system.
- **Increased Quality:** This feature increases in quality is largely a by-product of this program reuse.
- **Maintainable:** The objects can be maintained separately, making locating and fixing problems easier.
- **Scalable:** The object oriented applications are more scalable than structured approach.
- **Modularity:** The OOD systems are easier to modify.
- **Modifiability:** It is easy to make minor changes in the data representation or the procedures in an object oriented program.
- **Client/Server Architecture:** It involves the transmission of messages back and forth over a network.

Overview Of Unified Approach



- The UML (Unified Modelling Language) is a set of notations and conventions used to describe and model an application.
- The Unified Approach (UA) specifies the tasks or steps to develop an application.
- **The UA consists of the following concepts:**
 - ☐ Usecase driven approach
 - ☐ Utilizing the UML for modelling
 - ☐ Object oriented analysis
 - ☐ Object oriented design
 - ☐ Repositories of reusable classes and maximum reuse
 - ☐ The layered approach
 - ☐ Incremental development and prototyping
 - ☐ Continuous testing

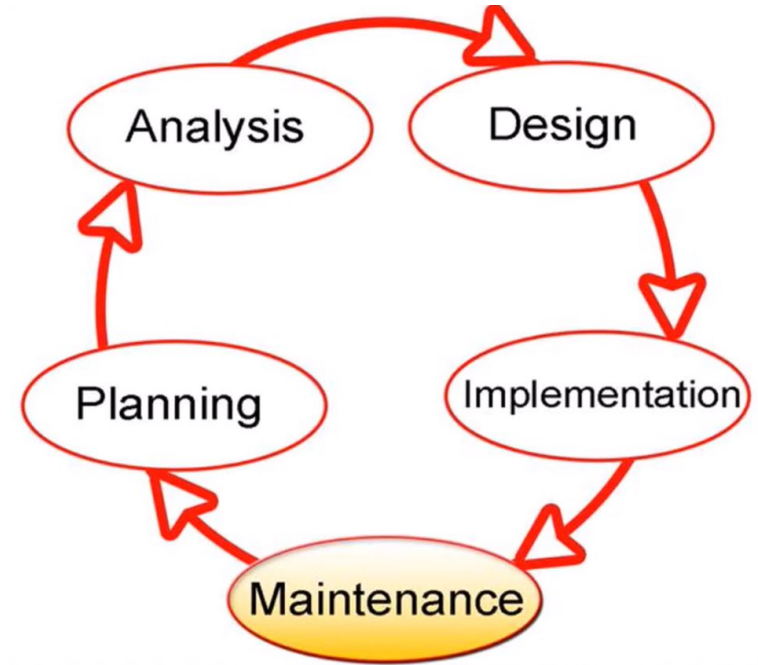


VIT[®]
B H O P A L
www.vitbhopal.ac.in

Object Oriented Systems Development Life Cycle

Object Oriented Systems Development Life Cycle

- The object-oriented life cycle model considers 'objects' as the basis of the software engineering process.
- The development team starts by observing and analysing the system they intend to develop before defining the requirements.
- Once the process is over, they focus on identifying the objects of the system. Now, an object could be anything; it can have a physical existence like a customer, car, etc. An object also constitutes intangible elements like a process or a project.



Object Oriented Systems Development Life Cycle



Advantages of Object-Oriented Life Cycle Model

Apart from enhancing the system performance, object-oriented programming offers some advantages such as:

- Since it is data-focused and easy to work with problem domains.
- It uses encapsulation and data hiding process that allows a developer to build tamper-proof systems.
- It enables software modularity, making it easier to manage and maintain complex software.
- It allows developers to create new modules using existing models, saving time and development cost of organizations.

Object Oriented Methodologies



- Object oriented methodologies are set of methods, models, and rules for developing systems.
- Modelling can be done during any phase of the software life cycle.
- A model is a an abstraction of a phenomenon for the purpose of understanding the methodologies.
- Modelling provides means of communicating ideas which is easy to understand the system complexity

Object-Oriented Methodologies are widely classified into three

- **Rumbaugh Methodology**
- **Booch Methodology**
- **Jacobson Methodology**

Object Oriented Methodologies



Object-Oriented Methodologies are widely classified into three

- **Rumbaugh Methodology**
 - **Booch Methodology**
 - **Jacobson Methodology**
-
- The Rumbaugh et al. method is well-suited for describing the object model or static structure of the system.
 - The Jacobson et.al method is good for producing user-driven analysis models
 - The Booch method detailed object-oriented design models

Rumbaugh Methodology (OMT)



- The Rumbaugh methodology also known as OMT (Object Modelling Technique)
- It is an approach used to develop manageable object-oriented systems and host object oriented programming.
- The purpose is to allow for class attributes, methods, inheritance, and association to be easily expressed.
- OMT is used in the real world for software modelling and designing.

According to Rumbaugh, there are several main reasons to utilize this modelling approach.

- ✓ One is to simulate entities before constructing them and another is to make communication with customers easier. Additionally, it help to reduce complexity through visualization.

Rumbaugh Methodology (OMT)



OMT consists of four stages:

- **Analysis:** Assigns an object, dynamic and functional model to the design. Determines important properties and domain.
- **Systems Design:** Outlines the basic systems structure of the program. Accounts for data storage and concurrency among other things.
- **Object Design:** Classifies objects and determines operations and data structures. Inheritance and different associations are also checked.
- **Implementation:** Conveys design through code.

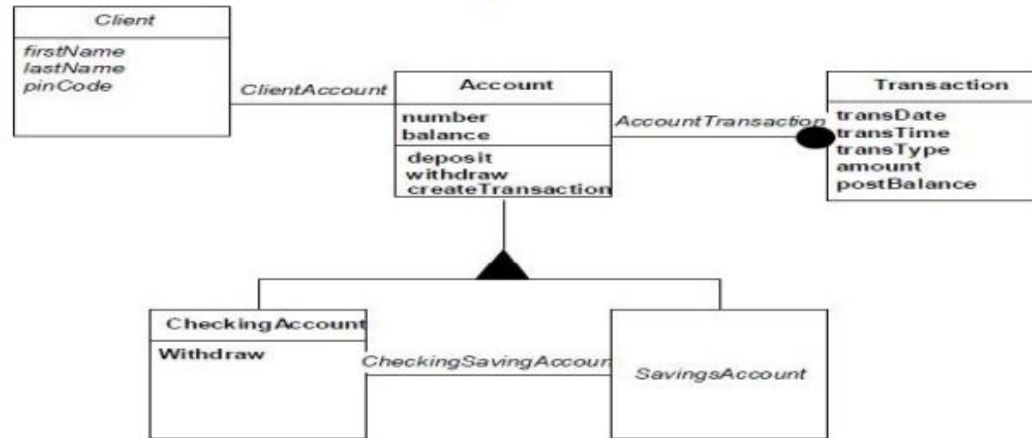
OMT is always broken down into three separate parts. These parts are:

- **Object Model:**
- **Dynamic Model:**
- **Functional Model:**

Rumbaugh Methodology (OMT)

Object Model:

- It describes the structure of objects in the system : their identity, relationships to other objects, attributes and operations.
- It contains classes interconnected by Association lines. Each Association line represents a set of links from the objects of one class to the objects of another class.

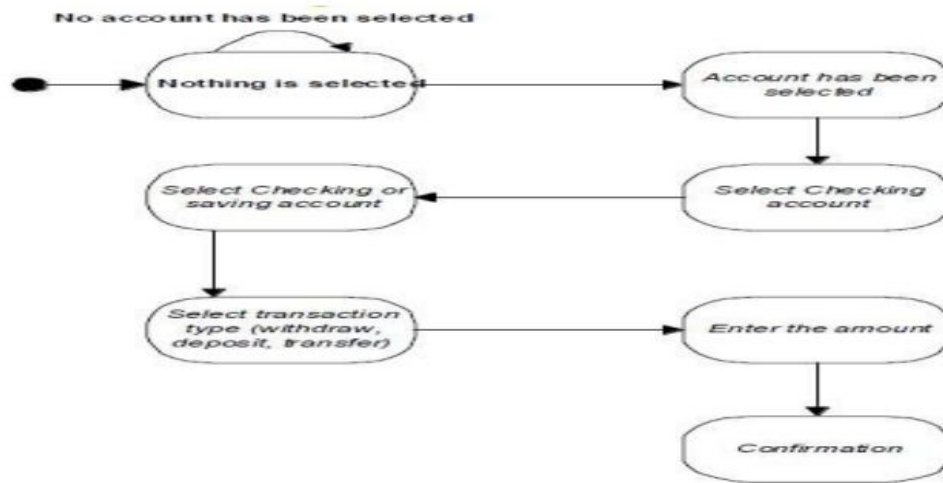


Example of an object model

Rumbaugh Methodology (OMT)

Dynamic Model:

- It provides detailed and comprehensive dynamic model, which depicts states, transitions, events and actions.
- It is a network of states and events and actions. Each state receives one or more events at which it makes the transition to the next step.

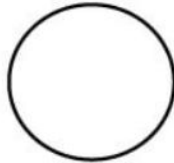


Example of a state transition for ATM Transaction

Rumbaugh Methodology (OMT)

Functional Model:

- Concerns itself with data flows, data storage, constraints, and processes

Symbol	Name	Function
	Data flow	Used to Connect Processes to each other, to sources or Sinks; the arrow head indicates direction of data flow.
	Process	Performs Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

Rumbaugh Methodology (OMT)



Notation/Diagramming Technique

- In OMT, abstract or concrete classes are represented as rectangles. Italicized font indicates abstraction while a normal font represents a concrete class.
- Lines indicate an association. There can be different multiplicities of association:
 - A regular line indicates exactly one association
 - Numbers above the line will indicate the multiplicity of associations
 - A line with an open circle indicates optional multiplicity
 - A line with a filled circle indicates many multiplicities
 - A line with a diamond indicates an aggregation
- **Note:** Multiplicity indicates the number of objects that can participate in a relationship or the number of instances of one class as it relates to another.

CONCRETE

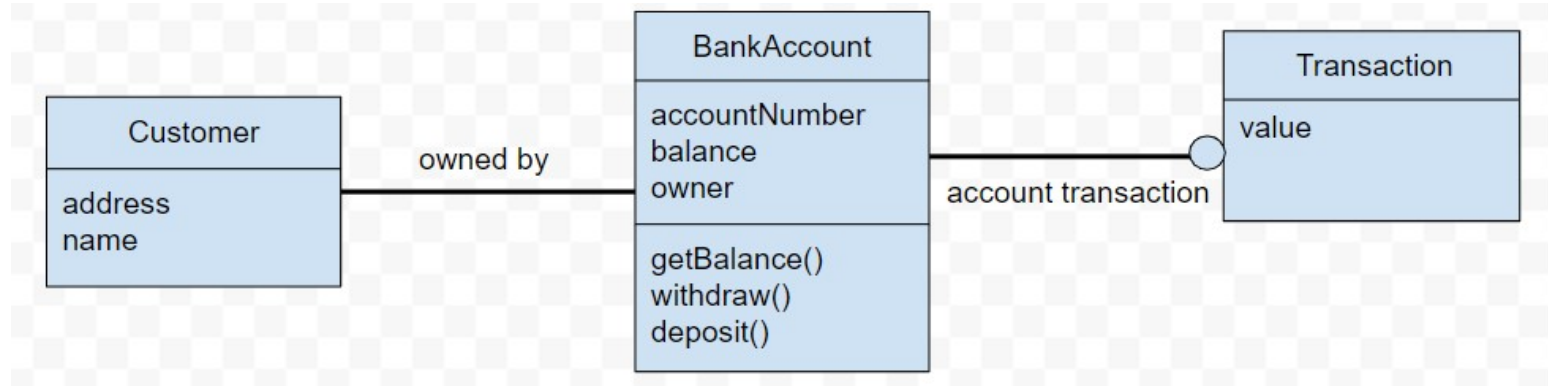
ABSTRACT

Rumbaugh Methodology (OMT)

Example:

Below is a simple example of a bank account. We've described the relationship and process of simple transactions between customers and bank accounts. We've also listed the instances and methods (only in the Bank Account class). And we have optional multiplicity of association between our transaction and bank account.

Take a look.



Booch Methodology



- It is a widely used object oriented method that helps us to design the system
- using object paradigm.
- The Booch methodology covers the analysis and design phases of systems development.
- The methodology details six different models/diagrams:
 - Class diagrams
 - Object diagrams
 - State transition diagrams
 - Module diagrams
 - Process diagrams
 - Interaction diagrams

Booch Methodology



Macro process:

- **Conceptualization:** Establish requirements taking into account the perspective of the customer
- **Analysis:** Develop a model by defining object classes, their attributes, methods, and inheritance. Include associations, the dynamic part of a model.
- **Design:** Develop a structure/architecture where logical and physical details are discussed
- **Evolution:** As it relates to the implementation
- **Maintenance:** Maintenance following the delivery of the product

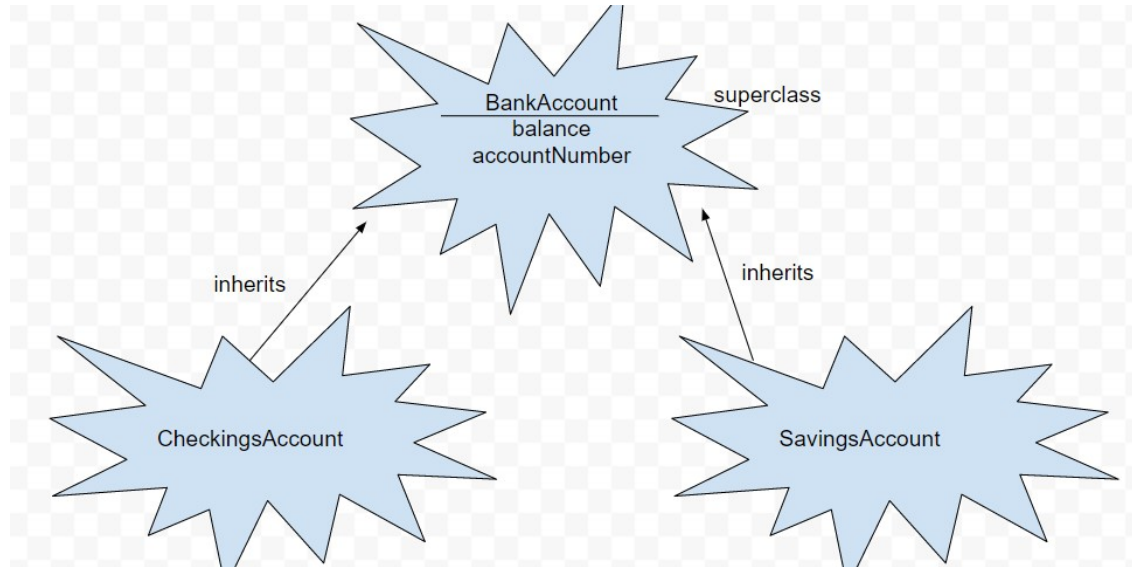
Micro process

- Identify classes and objects
- Identify semantics as it relates to programming
- Identify interactions/relationships

Booch Methodology

Example

Below is a diagram illustrating a booch diagram as it relates to the types of bank accounts. We have a superclass BankAccount and we've listed its attributes below. As you can see, our classes SavingsAccount and CheckingsAccount inherit the superclass.



Jacobson Methodology (OOSE)



- The Jacobson methodology, also known as Object-Oriented Software Engineering (OOSE).
- It is a method used to plan, design, and implement object-oriented software.
- The method is broken down into five parts: a set of requirements, an analysis, a design, an implementation, and a testing model.
 - **Requirements:** Create problem domain object diagram (as they satisfy requirements) and specifies use case diagrams
 - **Analysis:** Analysis diagrams (these are similar to the ones we covered in the other methods)
 - **Design:** State transition diagrams and interaction diagrams (these are similar to the ones we covered in the other methods)
 - **Implementation:** Implementation of the model
 - **Testing Model:** Testing of the model

Jacobson Methodology (OOSE)



- Uniquely, the methodology or OOSE utilizes use cases in its design.
- Use cases help us to understand that how we want to design our system; more specifically, they are scenarios for helping us understanding the requirements of our system.
- We can classify a use case as an interaction between a user and the system.
- A use case must contain these several elements:
 - The how and when the use case begins and ends
 - Governs interactions between the various actors
 - Data storage and data usage
 - Exceptions
 - How the constraints of the problem domain are handled

Jacobson Methodology (OOSE)

Example

